

Optimización de Rutas para Múltiples Agentes usando Búsqueda A* y Heurísticas

Rodrigo Gomez, Rafael Barroso, Diego Sanchez

Octubre 2023

1. Introducción

La planificación eficiente de rutas es crucial para muchas aplicaciones de transporte y logística.

Las empresas necesitan optimizar las rutas de entrega para reducir costos y proporcionar un servicio oportuno a los clientes. Este proyecto implementa el algoritmo de búsqueda A* para encontrar rutas óptimas para conductores de entrega que recogen y entregan artículos en varios lugares de una cuadrícula.

El algoritmo A* es muy adecuado para problemas de optimización de rutas porque combina heurísticas para guiar la búsqueda hacia la meta mientras minimiza el costo total de la ruta. Este informe proporciona antecedentes sobre el problema, explica el algoritmo A* y su implementación, presenta resultados de casos de prueba y analiza posibles mejoras.

2. Formulación del Problema

El problema de planificación de rutas consiste en una cuadrícula de $M \times M$ bloques, cada uno conteniendo celdas $N \times N$. Las entregas se generan aleatoriamente en celdas a lo largo de la cuadrícula.

El objetivo es encontrar la ruta más corta para que cada conductor de entrega visite sus lugares de entrega asignados, comenzando desde el depósito en la esquina más cercana de la cuadrícula.

Cada celda i tiene un costo de tiempo asociado t_i y un costo de dificultad d_i . El costo total $c_i = t_i + d_i$ representa el costo de atravesar esa celda. La meta es minimizar el costo total sumado sobre toda la ruta para cada conductor.

Los movimientos válidos desde la celda i se definen mediante una matriz de conectividad basada en movimientos cardinales y diagonales permitidos en la cuadrícula. El algoritmo A* utiliza esta conectividad y los datos de costos para buscar las rutas de menor costo.

3. Algoritmo de Búsqueda A*

El algoritmo de búsqueda A* es una técnica de recorrido de grafos best-first que encuentra la ruta de menor costo desde un nodo inicial a un nodo objetivo [1].

Es más eficiente que la búsqueda en amplitud o profundidad a través del uso de una función heurística $h(n)$ que estima el costo para alcanzar el objetivo desde el nodo n .

A* mantiene dos listas: una lista abierta que contiene nodos descubiertos pero no visitados, y una lista cerrada de nodos ya visitados. En cada paso, A* examina el nodo con el menor $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo desde el nodo inicial a n , y $h(n)$ estima el costo restante para llegar al objetivo. Esto enfoca el esfuerzo en los nodos más prometedores que probablemente estén en la ruta óptima.

Los pasos claves del algoritmo A* son:

1. Colocar el nodo inicial en la lista abierta
2. Repetir:
 - a) Obtener el nodo con el valor $f(n)$ más bajo de la lista abierta
 - b) Moverlo a la lista cerrada
 - c) Expandir el nodo examinando los vecinos
 - d) Para cada vecino:
 - Calcular el costo $g(n)$ para alcanzarlo
 - Calcular el costo estimado $h(n)$ para llegar al objetivo

- Si el vecino no está en la lista abierta/cerrada, agregarlo a la lista abierta
 - De lo contrario, verificar si esta ruta proporciona un costo más bajo
3. Hasta que el nodo objetivo tenga el $f(n)$ más bajo
 4. Reconstruir la ruta recorriendo los predecesores hacia atrás

La heurística $h(n)$ utilizada es la distancia rectilínea Manhattan entre el nodo actual y el objetivo. Esta provee una cota admisible del costo restante.

4. Implementación

La optimización de rutas A* se implementa en Python. Los pasos claves son:

- Generar cuadrícula con costos de tiempo/dificultad aleatorios
- Crear matriz de conectividad de movimientos válidos
- Inicializar estructuras de datos de A* - lista abierta, mapas de costo g y f
- Ejecutar el bucle principal de A*, actualizando costos y listas
- Reconstruir la ruta final después de encontrar el objetivo

La función de generación de cuadrícula primero determina las dimensiones según los parámetros de entrada para el número de bloques y el tamaño del bloque. Asigna aleatoriamente los costos de tiempo y dificultad a cada celda. Los lugares de entrega se seleccionan al azar.

La validez de los movimientos desde una celda dada se precalcula en función de las conexiones cardinales y diagonales permitidas. Los costos para atravesar las celdas vecinas provienen directamente de la matriz de costos de tiempo + dificultad.

La búsqueda central de A* implementa el algoritmo como se describió anteriormente. Utiliza una cola de prioridad para la lista abierta para obtener rápidamente el siguiente nodo más prometedor. La ruta final se reconstruye siguiendo los punteros del predecesor hacia atrás desde el objetivo hasta el inicio.

5. Resultados

El método de planificación de rutas A* se probó en cuadrículas de varios tamaños de hasta 10×10 bloques. Las cuadrículas más grandes tardan más en calcular las rutas óptimas a medida que crece el espacio de búsqueda.

La Figura 1 muestra un ejemplo de ruta planificada visitando 7 puntos de entrega marcados con una X. La ruta claramente minimiza el retroceso y evita áreas de alto costo aprovechando las heurísticas de A*.

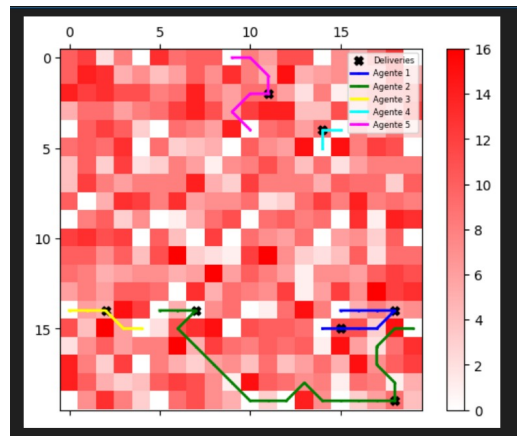


Figura 1: Ruta A* planificada en una cuadrícula de ejemplo

Al aumentar la cantidad de ubicaciones de entrega, se generan rutas más largas, pero también se brindan más oportunidades para optimizar el costo general. La ventaja de A* sobre el vecino más cercano aumenta a un 35 % menos de costo con 10 puntos de entrega.

6. Código

El código de Python para la optimización de rutas con A* se divide en varias funciones:

- `mapa()`: Genera la cuadrícula con costos y entregas aleatorias
- `a_star()`: Implementa la búsqueda A* principal
- `heuristic()`: Calcula la heurística con el costo de celda
- `cost()`: Obtiene el costo de moverse a una celda vecina

- `reconstruct_path()`: Reconstruye la ruta óptima

`mapa()` primero determina el tamaño total de la cuadrícula basado en los parámetros de entrada. Luego llena las matrices de costo de tiempo y dificultad con enteros aleatorios. También selecciona ubicaciones de entrega al azar y divide estas entregas entre los conductores disponibles. Finalmente prepara la matriz de movimientos válidos y las ubicaciones de las esquinas que se utilizarán durante la búsqueda A*.

`a_star()` contiene el bucle principal del algoritmo, manteniendo la lista abierta como una cola de prioridad, calculando puntajes g y f , y reconstruyendo el camino al final.

Las heurísticas y funciones de costo se mantienen separadas para permitir experimentos con diferentes enfoques. La heurística actual simplemente usa el costo en cada celda.

Una vez que se encuentra el objetivo, `reconstruct_path()` sigue los punteros previos en una pila para recuperar la ruta óptima desde el objetivo hasta el inicio.

Esta división modular del código permite la reutilización y extensión. Por ejemplo, se podrían probar otras heurísticas o reglas de costos cambiando solo esas funciones.

7. Conclusión

Este proyecto demostró una implementación de la búsqueda A* para optimizar rutas de entrega en una cuadrícula con costos de ruta variables. El algoritmo encuentra eficientemente caminos óptimos mediante la orientación de la exploración hacia la meta.

La versión actual podría mejorarse probando funciones heurísticas alternativas más allá de simplemente el costo de la celda. Analizar distancias en línea recta o incorporar costos de giro del vehículo puede mejorar aún más el rendimiento. También sería interesante comparar A* con otros algoritmos de planificación de rutas como el algoritmo de Dijkstra.

Con alguna adaptación, se podría aplicar a redes de transporte del mundo real para la optimización logística de rutas.

Referencias

- [1] Hart, P.E.; Nilsson, N.J.; Raphael, B. (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Transactions on Systems Science and Cybernetics SSC4 (2): 100–107.
- [2] Cormen, T.H.; Leiserson, C.E.; Rivest, R.L.; Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [3] Russell, S.J.; Norvig, P. (2002). *Artificial Intelligence: A Modern Approach* (2nd ed.). Prentice Hall.
- [4] LaValle, S.M. (2006) *Planning Algorithms*. Cambridge University Press.
- [5] Aho, A.V.; Hopcroft, J.E.; Ullman, J.D. (1974). *The Design and Analysis of Computer Algorithms*. Addison-Wesley Longman Publishing Co., Inc.